

Gestion Sécurisée des Secrets avec SecretStore

Objectif

Éradiquer définitivement les mots de passe en clair dans les scripts PowerShell. Vous allez configurer un coffre-fort local, le paramétrer pour un usage automatisé (sans intervention humaine, type CI/CD), y injecter des secrets, et concevoir un script capable de les récupérer à la volée.

Exercice 1 : Initialisation et Configuration du Coffre

(Note : Les modules Microsoft.PowerShell.SecretManagement et Microsoft.PowerShell.SecretStore sont déjà installés sur vos postes).

1. Enregistrement du coffre :

Déclarez votre coffre-fort local auprès du gestionnaire en le nommant "CoffreDevOps". Définissez-le comme le coffre par défaut.

Aide syntaxique :

```
Register-SecretVault -Name "CoffreDevOps" -ModuleName  
"Microsoft.PowerShell.SecretStore" -DefaultVault
```

2. Configuration pour l'automatisation (Unattended) :

Puisque nous simulons un environnement d'exécution automatisé (type tâche planifiée ou pipeline CI/CD), nous ne voulons pas qu'une fenêtre contextuelle demande un mot de passe maître.

Configurez le coffre pour désactiver l'authentification par mot de passe et les invites de confirmation.

Aide syntaxique :

```
Set-SecretStoreConfiguration -Authentication None -Interaction None  
-ConfirmPrompt:$false
```

(Le coffre utilisera désormais l'API cryptographique locale de votre session Windows pour se déverrouiller automatiquement).

Exercice 2 : Enregistrement des Données Sensibles

Vous allez stocker deux types de secrets différents dans votre coffre.

1. Un Jeton d'API (Chaîne de texte brute) :

Créez un secret nommé "Token_Supervision" contenant la valeur fictive "abc123_XYZ_789".

Aide syntaxique :

Set-Secret -Name "Token_Supervision" -Secret "abc123_XYZ_789"

2. Un Compte de Service ([PSCredential]) avec Métadonnées :

- Générez un objet d'identification via la commande `$MonCred = Get-Credential`. (Entrez "Admin_Sauvegarde" comme utilisateur et un mot de passe de votre choix).
- Stockez cet objet dans le coffre sous le nom "Compte_Sauvegarde".
- Ajoutez-y des métadonnées sous forme de table de hachage pour préciser le contexte : `-Metadata @{ Application = "Veeam" ; Environnement = "Production" }`.

Exercice 3 : Audit Sans Divulgation (Le rôle du RSSI)

Vous devez vérifier ce que contient le coffre sans pour autant charger les mots de passe en mémoire vive.

1. Utilisez la commande d'information des secrets :
`Get-SecretInfo`
2. Observez le résultat :
 - Quel est le type reconnu pour "Token_Supervision" ?
 - Quel est le type reconnu pour "Compte_Sauvegarde" ?
 - Voyez-vous les métadonnées que vous avez attachées au compte de sauvegarde ?
 - *Constatez que les mots de passe eux-mêmes ne sont jamais affichés à l'écran.*

Exercice 4 : Consommation dans un Script de Production

Créez un nouveau fichier de script nommé `C:\Audit\Test-API.ps1`. Vous allez écrire un code qui extrait et utilise le jeton d'API de manière sécurisée.

1. Copiez-collez le code suivant :

```
Write-Host "Démarrage de la sonde de supervision..." -ForegroundColor Cyan

try {
    # 1. Extraction du secret en texte clair pour pouvoir l'envoyer dans l'en-tête HTTP
    $Jeton = Get-Secret -Name "Token_Supervision" -AsPlainText -ErrorAction Stop

    # 2. Utilisation du secret
    Write-Host "Le jeton récupéré depuis le coffre a une longueur de $($Jeton.Length) caractères."
    Write-Host "Simulation de la requête Invoke-RestMethod..." -ForegroundColor Yellow

    # (Ici se trouverait le vrai code d'appel d'API)
}
```

```
catch {  
    Write-Host "Échec d'authentification : $($_.Exception.Message)" -ForegroundColor Red  
}  
finally {  
    # 3. Règle d'or de sécurité : Purger la variable sensible de la mémoire RAM  
    $Jeton = $null  
    Write-Host "Variable mémoire purgée. Fin du script." -ForegroundColor Green  
}
```

2. **Exécutez votre script.**

Constatez que le script accède au secret sans vous poser de question, et qu'aucun mot de passe n'est écrit dans votre fichier texte .ps1.

3. **Nettoyage :**

Supprimez le secret "Token_Supervision" de votre coffre à l'aide de la commande Remove-Secret. Relancez votre script Test-API.ps1 pour valider que la capture d'erreur (le bloc catch) fonctionne correctement lorsque le secret a disparu.